PROJET ACADÉMIQUE — RAPPORT FINAL

CAMPUS EVENT

Conception et réalisation d'un système d'information de gestion d'événements universitaires

---

Étudiant :	**Stephen AKUGBE**
Module :	**Introduction aux Systèmes d'Information**
Technologies :	**Python, Flask, REST API, HTML5/CSS3/JS, UML**
Date :	**Année Universitaire 2026 – 2027**
Statut des tests :	**10 / 10 Tests Passés (100%)**

1. Présentation du Système et du Besoin Métier

Dans le cadre de ses activités pédagogiques et associatives, l'université organise de multiples manifestations : conférences académiques, forums entreprises, ateliers techniques et hackathons. Auparavant, la gestion de ces événements reposait sur une mosaïque de formulaires indépendants et de fichiers tableurs dispersés.

Cette gestion non centralisée présentait des dysfonctionnements majeurs :

- **Dispersion de l'information** : absence de base unique et risques d'incohérence des plannings.
- **Doubles inscriptions** : un même étudiant pouvait s'inscrire plusieurs fois par inadvertance.
- **Dépassement des jauges** : absence de temps réel sur les places disponibles et saturation des salles.
- **Manque de visibilité** : difficulté pour les organisateurs et l'administration de suivre les participants.

> **Objectif métier de CAMPUS EVENT :** Centraliser sur une plateforme unifiée et sécurisée la consultation du catalogue d'événements, le traitement automatisé des demandes d'inscription et la régulation des capacités en temps réel.

2. Analyse du Système d'Information (Partie 1)

2.1 Identification des acteurs et de leurs rôles

Trois profils d'acteurs interagissent avec le système :

Acteur	Rôle principal	Informations consultées	Informations créées / modifiées
Étudiant	Consulte le catalogue et postule aux événements.	Liste des événements, places restantes, description, date/lieu, état de ses inscriptions.	Création d'une demande d'inscription (statut initial `pending`).
Organisateur	Planifie les manifestations et gère la jauge de participants.	Fiches événements créés, liste des inscrits par événement, taux de remplissage.	Publication d'événements (capacité, date), validation du statut (`accepted` / `rejected`).
Administration	Supervise globalement le système et assure le reporting.	Statistiques globales, historique étudiant, volumétrie des flux.	Gestion des comptes étudiants, paramétrages transversaux de l'université.

2.2 Identification des processus métier

Le système orchestre trois processus métier essentiels :

Processus	Objectif métier	Étapes principales	Acteur(s)
P1 : Inscription à un événement	Permettre à un étudiant de postuler en vérifiant les jauges et l'unicité.	1. Consultation catalogue 2. Sélection événement 3. Contrôle existence étudiant & événement 4. Détection doublon 5. Vérification places restantes 6. Création statut `pending`	Étudiant, Système
P2 : Création d'événement	Mettre à disposition une nouvelle activité sur le campus.	1. Saisie des métadonnées (titre, date, lieu, jauge) 2. Contrôle de validité des champs 3. Enregistrement et publication dans le catalogue public	Organisateur
P3 : Traitement des inscriptions	Réguler les admissions et notifier les étudiants.	1. Consultation des demandes en attente 2. Décision d'acceptation ou de rejet 3. Mise à jour statut (`accepted` / `rejected`) 4. Recalcul des places restantes	Organisateur

2.3 Matrice des informations du système

Catégorie	Informations principales	Qui les crée ?	Qui les consulte ?
Étudiants	Identifiant unique (`id`), Nom, Prénom, Email universitaire, Filière.	Administration	Étudiant, Organisateur, Administration
Événements	Identifiant (`id`), Titre, Description, Date & Heure, Lieu, Capacité totale.	Organisateur	Étudiants, Organisateurs, Administration
Inscriptions	Identifiant (`id`), `student_id` , `event_id` , Date demande, Statut (`pending` , `accepted` , `rejected` , `waitlisted`).	Étudiant (création) / Organisateur (statut)	Étudiant, Organisateur, Administration
Pilotage / Statistiques	Taux de remplissage, nombre d'inscriptions totales, places restantes par événement.	Système (calculé)	Administration, Organisateurs

2.4 Cartographie des flux d'information

Le schéma ci-dessous modélise les échanges d'information entre les acteurs et CAMPUS EVENT :

Schéma des flux d'information

Figure 1 : Cartographie des flux d'information (Qui envoie quoi à qui ?)

3. Architecture du Système (Partie 2)

Le système adopte une **architecture 3-tiers** classique garantissant un découplage fort entre la présentation visuelle, l'application des règles métier et la persistance des données.

Niveau (Tier)	Rôle	Traitements & Responsabilités	Technologies
1. Présentation	Interface Utilisateur	Affichage ergonomique, requêtes asynchrones `fetch()` vers l'API, saisie du formulaire, manipulation dynamique du DOM.	HTML5, CSS3, Vanilla JavaScript
2. Logique Métier	Serveur d'application	Routage REST, vérification des règles métier (doublons, saturation jauge), gestion des statuts HTTP (200, 201, 400, 404, 409).	Python 3, Flask, Flask-CORS
3. Données	Persistance	Stockage centralisé des listes d'étudiants, d'événements et d'inscriptions. Intégrité des identifiants et des relations.	Listes Python en mémoire (SQLite en bonus)

> **Règle d'or de l'architecture :** Le frontend ne dialogue *jamais* directement avec le tier Données. Toutes les interactions transitent impérativement par les routes de l'API REST Flask.

Figure 2 : Architecture 3-tiers de la solution CAMPUS EVENT

4. Conception de l'API REST (Partie 3)

4.1 Ressources REST identifiées

- `Student` (URI : `/students`) : Représente les étudiants inscrits à l'université.
- `Event` (URI : `/events`) : Représente les événements organisés sur le campus.
- `Registration` (URI : `/registrations`) : Représente le lien d'inscription liant un étudiant à un événement avec son statut.

4.2 Tableau des Endpoints et codes succès attendus

Besoin métier	Méthode HTTP	Endpoint URI	Code succès attendu
Consulter tous les événements	`GET`	`/events`	`200 OK`
Consulter un événement précis	`GET`	`/events/<id>`	`200 OK`
Consulter les étudiants	`GET`	`/students`	`200 OK`
Créer une inscription	`POST`	`/registrations`	`201 Created`
Consulter les inscriptions	`GET`	`/registrations`	`200 OK`
Consulter une inscription précise	`GET`	`/registrations/<id>`	`200 OK`
Modifier le statut d'une inscription	`PATCH`	`/registrations/<id>`	`200 OK`

5. Règles Métier et Traitement des Codes HTTP (Partie 4)

Chaque requête transmise au serveur subit un pipeline de vérification strict afin de garantir l'intégrité des données avant enregistrement :

Règle métier	Comportement attendu & Message d'erreur	Code HTTP attendu
Champs obligatoires absents	Refus si le JSON reçu ne comporte pas simultanément `student_id` et `event_id`.	`400 Bad Request`
Étudiant inexistant	Refus si le `student_id` n'existe pas dans la table des étudiants.	`404 Not Found`
Événement inexistant	Refus si le `event_id` ne correspond à aucun événement existant.	`404 Not Found`
Événement complet	Refus si le nombre d'inscriptions déjà validées ou en attente atteint la capacité maximale.	`409 Conflict`
Double inscription	Refus si l'étudiant possède déjà une inscription active pour ce même événement.	`409 Conflict`
Création réussie	Enregistrement avec génération d'identifiant et initialisation au statut `pending`.	`201 Created`
Modification de statut réussie	Mise à jour du statut vers `accepted`, `rejected` ou `waitlisted`.	`200 OK`

6. Campagne de Tests REST / Postman (Partie 5)

La conformité de l'API a été validée via les 10 tests unitaires et d'intégration définis dans le sujet. Tous les scénarios ont été exécutés avec succès (taux de réussite de 100%) :

#	Test exécuté	Résultat attendu	Résultat obtenu	Commentaire d'évaluation
1	`GET /events`	`200 OK`	`200 OK`	Retourne la liste complète avec places calculées.
2	`GET /events/1` (existant)	`200 OK`	`200 OK`	Détail de l'événement n°1 obtenu avec succès.
3	`GET /events/999` (inexistant)	`404 Not Found`	`404 Not Found`	Message : « Événement introuvable ».
4	`POST /registrations` valide	`201 Created`	`201 Created`	Inscription créée avec statut initial `pending`.
5	`POST` JSON incomplet	`400 Bad Request`	`400 Bad Request`	Champ `event_id` manquant correctement rejeté.
6	`POST` étudiant inexistant	`404 Not Found`	`404 Not Found`	Étudiant n°999 introuvable dans la base.
7	`POST` événement inexistant	`404 Not Found`	`404 Not Found`	Événement n°999 introuvable dans la base.
8	`POST` événement complet	`409 Conflict`	`409 Conflict`	Rejet d'une 3e inscription sur jauge de 2 places.
9	`POST` double inscription	`409 Conflict`	`409 Conflict`	Détection et blocage de la réinscription de l'étudiant 1.
10	`PATCH /registrations/1` statut	`200 OK`	`200 OK`	Statut mis à jour vers `accepted`.

7. Modélisation UML (Partie 6)

7.1 Diagramme de Cas d'Utilisation

Ce diagramme circonscrit la frontière du système et matérialise les fonctionnalités offertes à chaque acteur. L'Étudiant interagit principalement avec la consultation et la demande d'inscription, tandis que l'Organisateur administre le catalogue et valide les candidatures.



Figure 3 : Diagramme de Cas d'Utilisation UML du système CAMPUS EVENT

7.2 Diagramme d'Activité — « S'inscrire à un événement »

Le diagramme d'activité met en lumière le flot séquentiel et les points de décision critiques : validation des identifiants, vérification de l'unicité de l'inscription et disponibilité des places. Toute anomalie bifurque vers une terminaison en échec avec code HTTP approprié.



Figure 4 : Diagramme d'Activité UML — Processus d'inscription

7.3 Diagramme de Séquence — Inscription à un événement

Ce diagramme présente l'ordonnancement temporel des messages échangés entre les quatre participants clés : l'**Étudiant**, l'interface **Frontend**, le serveur **API Flask** et le gestionnaire de **Données**. Il détaille le scénario nominal aboutissant au code `201 Created` ainsi que le scénario alternatif d'événement saturé (code `409 Conflict`).

Figure 5 : Diagramme de Séquence UML avec scénario nominal et alternatif

7.4 Diagramme de Classes Métier

Le modèle de classes s'articule autour des entités `Student` et `Event` reliées par la classe associative `Registration`.

- **Association Student — Registration (1..1 à 0..*) :** Un étudiant peut réaliser 0 à plusieurs inscriptions. Chaque inscription appartient obligatoirement à un unique étudiant.
- **Association Event — Registration (1..1 à 0..*) :** Un événement peut recevoir 0 à plusieurs inscriptions. Chaque inscription concerne obligatoirement un unique événement.
- **Rôle de Registration :** Cette entité est indispensable car elle porte les attributs propres à la relation : identifiant de transaction, horodatage et statut d'avancement (`pending` / `accepted` / `rejected` / `waitlisted`).

Figure 6 : Diagramme de Classes UML et cardinalités

8. Réalisation de l'Interface Frontend (Partie 7)

L'interface utilisateur a été développée dans le fichier `index.html` en respectant les standards modernes du Web :

- **Appel dynamique `fetch()` :** Récupération asynchrone des événements dès le chargement de la page via `GET /events`.
- **Affichage dynamique des données :** Génération de cartes esthétiques indiquant pour chaque événement son titre, son lieu, sa date, sa capacité maximale et son **nombre de places disponibles calculé en temps réel**.
- **Gestion de la résilience réseau :** Détection et affichage d'une alerte claire si l'API Flask est inaccessible ou arrêtée.
- **Formulaire d'inscription interactif (Bonus) :** Sélection guidée d'un étudiant et d'un événement, soumission via `POST /registrations` en JSON, et retour visuel immédiat (badge vert de confirmation ou encadré rouge détaillant l'erreur 400, 404 ou 409).

9. Difficultés Rencontrées et Choix Réalisés

- **Résolution du blocage CORS :** Lors des premiers essais entre le navigateur et le serveur Flask, la politique de sécurité Same-Origin bloquait les requêtes `fetch()`. L'intégration de la bibliothèque `flask-cors` a résolu immédiatement ce problème sans affaiblir la sécurité du serveur.
- **Calcul dynamique vs statique des places :** Plutôt que de décrémenter une jauge statique dans l'objet événement, le système calcule à la volée le nombre de places restantes à partir des inscriptions au statut `pending` et `accepted`. Cela évite toute incohérence lors des rejets ou annulations d'inscriptions.
- **Uniformité des réponses d'erreur :** Tous les endpoints retournent systématiquement des objets JSON normalisés comportant la clé `error` afin de simplifier le parsing côté client JavaScript.

10. Conclusion et Perspectives

Le projet **CAMPUS EVENT** répond pleinement au cahier des charges fixé. Il démontre la maîtrise complète du cycle de vie d'un système d'information : de l'analyse métier initiale jusqu'au déploiement d'une API REST Flask testée et documentée, accompagnée d'un frontend réactif et d'une modélisation UML rigoureuse.

En perspective d'évolution, le système pourrait être enrichi par :

1. La persistance sur base relationnelle **SQLite** avec SQLAlchemy pour conserver l'historique lors des redémarrages.
2. Une file d'attente automatisée (FIFO) affectant directement les places libérées aux étudiants `waitlisted`.

3. L'ajout d'une authentification par jetons **JWT** différenciant les rôles Étudiant et Organisateur.